REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY COLLEGE OF SCIENCE
LEGAZPI CITY

Tap Control: Water Supply District Management System

TECHNICAL DOCUMENTATION
WEB SYSTEMS AND TECHNOLOGIES

SUBMITTED BY:
ASAYTUNO, MARK DAVE
LOPERA, JOSHUA
LUNAS, ALDWIN JUDE
SAPIN, JAYSON VICTOR

AY 2025-2026

1. PROJECT OVERVIEW

1.1 Project Title

TapControl: Water Supply District Management System

1.2 Original Authors/Source

This project was conceived and developed entirely from scratch by the project team. There is no original published paper, external repository, or prior codebase that this system is based on. TapControl is an original Web System work.

1.3 Module Scope

TapControl is a full-stack web application designed to digitize and automate the day-to-day operations of a barangay-level water supply organization. The system covers the following functional modules:

- District Management - track service areas, usage percentages, and capacity thresholds
- Consumer Management - register and manage household/commercial water subscribers
- Staff Management - maintain personnel records and district assignments
- Meter Readings - log monthly meter data and auto-compute water consumption
- Billing - generate bills using a tiered rate structure
- Payments - record payments and automatically mark linked bills as paid
- Service Requests - file and track consumer complaints (leaks, meter issues, new connections)
- Dashboard - a real-time summary panel showing key KPIs across all modules

1.4 Rationale

The inspiration for TapControl comes directly from a real-world problem observed in our barangay: the local water supply organization had no digital system and operated entirely on manual paper records, carbon-copy receipts, handwritten ledgers, and physical folders of consumer cards. This created several operational challenges:

- Billing errors from manual computation and transcription mistakes
- Delayed overdue detection because there was no automated status tracking
- Lost or degraded paper records with no backup mechanism
- No visibility into district-level usage or system load
- Staff had no centralized tool for service request tracking

TapControl was built to directly address these pain points by providing a centralized, browser-based platform that any staff member can use without specialized technical training.

1.5 System Gap

Because this project was built from the ground up with a specific barangay context in mind, it differs from generic utility billing systems in several meaningful ways:

Aspect	Generic Utility System	TapControl
Target Scale	City/municipality-wide (thousands of consumers)	Barangay-level (hundreds of consumers)
Setup Complexity	Enterprise installation required	XAMPP + Node.js; runs on any barangay office PC
District Auto-Status	Manual status flags	Auto-derived from usage % and consumer % thresholds
Cost	Subscription or licensing fees	Free, open-source stack

2. TECHNOLOGY STACK

Layer	Technology
Frontend Framework	Vanilla HTML5 + JavaScript (ES6+)
Styling	Bootstrap 5 + Custom CSS
Backend Framework	Node.js / Express 4
Database	MySQL 8.0 (via XAMPP)
Additional Libraries	cors, dotenv, mysql2, nodemon
IDE / Development Tools	Visual Studio Code, Claude AI

2.1 Project Structure

```
tapcontrol/
├── frontend/
│   ├── index.html          ← Dashboard
│   ├── consumers.html
│   ├── meters.html
│   ├── districts.html
│   ├── staff.html
│   ├── billing.html
│   ├── payments.html
│   ├── servicerequest.html
│   ├── login.html
│   ├── css/                ← components, layout, utilities
│   └── js/                 ← per-page JS modules
└── backend/
    ├── server.js           ← Express entry point
    ├── .env                ← DB credentials
    ├── db/
    │   ├── connection.js   ← MySQL pool
    │   └── tapcontrol.sql  ← Schema + seed data
    └── routes/
        ├── dashboard.js    billing.js   payments.js
        ├── consumers.js    meters.js    districts.js
        ├── staff.js        serviceRequests.js
```

3. DATABASE DESIGN

3.1 Entity-Relationship Overview

The TapControl database (tapcontrol) consists of 7 tables. The central entity is consumers, which links outward to all transactional tables. Districts serve as a geographic grouping entity shared by consumers, staff, and meter readings.

Table	Description	Primary Key	Foreign Keys
districts	Service area zones with capacity limits and auto-derived status	id (VARCHAR 10, e.g. D01)	

consumers	Water subscribers; auto-assigned consumer_id and meter_no on creation	consumer_id (VARCHAR 10, e.g. C-001)	district_id → districts.id
staff	Organization personnel with roles and district assignments	staff_id (VARCHAR 10, e.g. ST-001)	district_id → districts.id
meter_readings	Monthly meter logs; consumption is a GENERATED column (curr − prev)	id (VARCHAR 10, e.g. MR-001)	consumer_id → consumers, district_id → districts
billing_records	Generated bills with tiered amount_due; status: Unpaid / Paid / Overdue	id (VARCHAR 10, e.g. BL-001)	consumer_id → consumers
payments	Payment transactions; recording a payment auto-sets linked bill to Paid	id (VARCHAR 10, e.g. PAY-001)	consumer_id → consumers, bill_id → billing_records
service_requests	Consumer complaints and requests (leaks, new connections, etc.)	id (VARCHAR 10, e.g. SR-001)	consumer_id → consumers

3.2 Relationship Summary

Parent Table	Child Table	Relationship	Description
districts	consumers	1 to Many	A district has many consumers.
districts	staff	1 to Many	A district has many staff members.
districts	meter_readings	1 to Many	A district has many meter readings.
consumers	meter_readings	1 to Many	A consumer has many meter readings.
consumers	billing_records	1 to Many	A consumer has many billing records.
consumers	payments	1 to Many	A consumer makes many payments.
consumers	service_requests	1 to Many	A consumer files many service requests.
billing_records	payments	1 to Many	A bill can be settled by one or more payments.

3.3 Data Dictionary

districts

Field Name	Data Type	Description
id	VARCHAR(10)	District ID (e.g. D01). Primary key.
name	VARCHAR(100)	Full name of the district.
usage_pct	DECIMAL(5,2)	Current water usage as a percentage (0–100%).
consumer_count	INT	Number of active consumers in the district.
max_capacity_m3	DECIMAL(10,2)	Maximum water capacity in cubic meters.
max_consumers	INT	Maximum allowed number of consumers.
status	ENUM	Derived status: Operational / Near Limit / Critical.
created_at	TIMESTAMP	Record creation timestamp.

consumers

Field Name	Data Type	Description
consumer_id	VARCHAR(10)	Consumer ID (e.g. C-001). Primary key.
district_id	VARCHAR(10)	References districts(id). ON UPDATE CASCADE / ON DELETE RESTRICT.
full_name	VARCHAR(150)	Full name of the consumer/subscriber.
address	VARCHAR(255)	Physical address of the consumer.
contact_number	VARCHAR(20)	Contact phone number.
meter_no	VARCHAR(30)	Unique meter number (e.g. MTR-00001). Auto-assigned on creation.
account_type	ENUM	Account class: Residential / Commercial / Industrial.

status	ENUM	Account status: Active / Inactive / Maintenance.

staff

Field Name	Data Type	Description
staff_id	VARCHAR(10)	Staff ID (e.g. ST-001). Primary key.
name	VARCHAR(150)	Full name of the staff member.
role	ENUM	Role: Meter Reader / Technician / Billing Officer / Supervisor / Admin.
district_id	VARCHAR(10)	References districts(id). ON DELETE SET NULL.
contact	VARCHAR(20)	Contact phone number.
status	ENUM	Employment status: Active / Inactive / On Leave.
date_hired	DATE	Date the staff member was hired.

meter_readings

Field Name	Data Type	Description
id	VARCHAR(10)	Reading ID (e.g. MR-001). Primary key.
meter_no	VARCHAR(30)	Meter number corresponding to the consumer's meter.
consumer_id	VARCHAR(10)	References consumers(consumer_id). ON DELETE CASCADE.
district_id	VARCHAR(10)	References districts(id). ON DELETE RESTRICT.
prev_reading	DECIMAL(10,2)	Previous meter reading value in cubic meters.
curr_reading	DECIMAL(10,2)	Current meter reading value in cubic meters.
consumption	DECIMAL(10,2)	Auto-computed: curr_reading −

		prev_reading. STORED generated column.
reading_date	DATE	Date the meter was read.
reader_name	VARCHAR(150)	Name of the staff member who took the reading.

billing_records

Field Name	Data Type	Description
id	VARCHAR(10)	Bill ID (e.g. BL-001). Primary key.
consumer_id	VARCHAR(10)	References consumers(consumer_id). ON DELETE CASCADE.
consumption	DECIMAL(10,2)	Water consumption in cubic meters for the billing period.
amount_due	DECIMAL(10,2)	Total amount due (consumption × 0.76). Rate-based computation.
due_date	DATE	Payment due date for the bill.
status	ENUM	Payment status: Unpaid / Paid / Overdue.
created_at	TIMESTAMP	Timestamp when the bill was generated.

payments

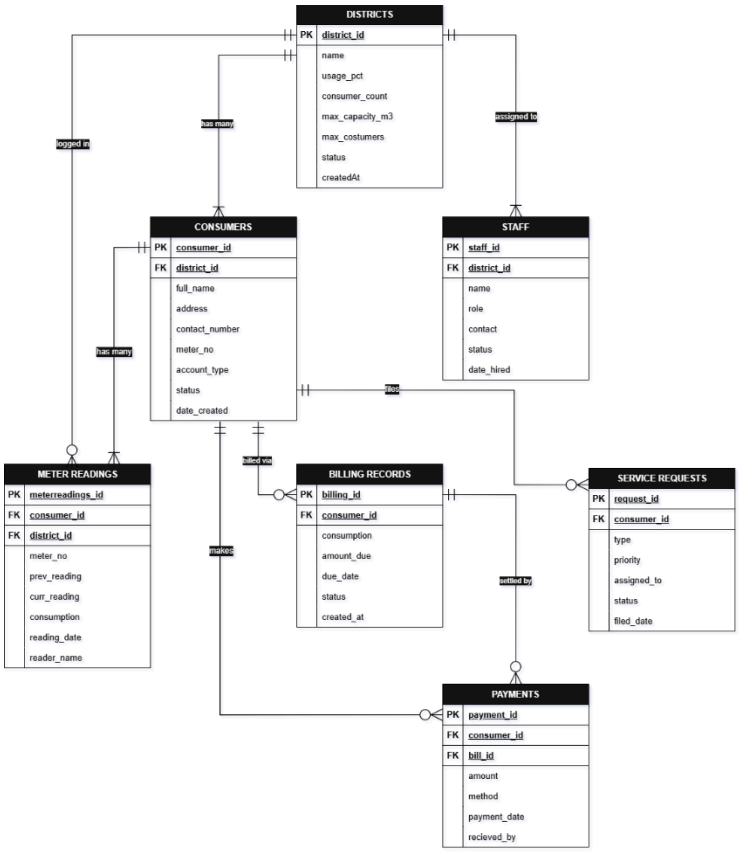
Field Name	Data Type	Description
id	VARCHAR(10)	Payment ID (e.g. PAY-001). Primary key.
consumer_id	VARCHAR(10)	References consumers(consumer_id). ON DELETE CASCADE.
bill_id	VARCHAR(10)	References billing_records(id). ON DELETE CASCADE.
amount	DECIMAL(10,2)	Amount paid by the consumer.

method	ENUM	Payment method: Cash / GCash / Bank Transfer.
payment_date	DATE	Date the payment was made.
received_by	VARCHAR(150)	Name of the staff who received the payment.

Service_request

Field Name	Data Type	Description
id	VARCHAR(10)	Request ID (e.g. SR-001). Primary key.
consumer_id	VARCHAR(10)	References consumers(consumer_id). ON DELETE CASCADE.
type	ENUM	Request type: Leak Repair / Meter Issue / New Connection / Disconnection / Other.
priority	ENUM	Priority level: Low / Medium / High.
assigned_to	VARCHAR(150)	Name of the staff assigned to handle the request.
status	ENUM	Request status: Open / In Progress / Resolved.
filed_date	DATE	Date the service request was filed.

3.4 Entity Relationship Diagram



4. MODULE FEATURES

Each module supports Create, Read, Update, and Delete operations through a REST API. Below is a concise breakdown of what each operation does, its validation rules, and how errors are handled.

4.1 Consumer Management

Registers and manages water subscribers. Each consumer is auto-assigned a Consumer ID and meter number.

Create: Requires full name, address, and contact number. Consumer ID (C-001) and meter number (MTR-00001) are auto-generated. Returns HTTP 400 if required fields are missing.

Read: Lists all consumers. Filterable by name, district, or account type. Shows 'No consumers found' if no match.

Update: Pre-filled edit form. Moving a consumer to another district auto-syncs both districts' consumer counts.

Delete: Confirmation: 'Delete this consumer? All linked records will also be removed.' Cascades to bills, payments, readings, and service requests.

4.2 Billing

Generates bills based on water consumption using the data Maynilad rate in 2025.

Create: Requires consumer and consumption (m³). Amount due is auto-calculated server-side — staff do not enter it manually. Status defaults to Unpaid.

Read: Lists all bills with consumer name, consumption, amount, due date, and status. Filterable by status (Unpaid / Paid / Overdue).

Update: Editing consumption recalculates the amount due automatically.

Delete: Confirmation: 'Delete this billing record? Linked payments will also be removed.

4.3 Payments

Records payment transactions and automatically marks the linked bill as Paid.

Create: Requires consumer, bill, and amount. Payment method defaults to Cash. On save, the linked bill status is instantly set to Paid.

Read: Lists all payments. Filterable by consumer name or payment method (Cash / GCash / Bank Transfer).

Update: Allows correction of amount, method, date, or assigned bill.

Delete: Confirmation: 'Delete this payment record?' Note: the linked bill status must be manually updated if needed.

4.4 Meter Readings

Logs monthly meter readings per consumer. Consumption is auto-computed by the database (current to previous).

Create: Requires meter number, consumer, and current reading. Current reading must be ≥ previous reading, otherwise returns: 'Current reading cannot be less than previous reading.'

Read: Lists all readings with consumption auto-shown. Filterable by consumer name or district.

Update: Not available — readings are final. Delete and re-submit to correct an error.

Delete: Confirmation: 'Delete this meter reading?' Permanently removes the entry.

4.5 Districts

Manages service area zones. Status (Operational / Near Limit / Critical) is auto-derived from live usage data.

Create: Requires district ID and name. Capacity defaults to 5,000 m³ and 500 consumers if not set.

Read: Shows each district's consumer count, usage %, and auto-computed status. Status thresholds: ≥90% → Critical, ≥75% → Near Limit.

Update: Name and capacity limits can be updated. District ID cannot be changed.

Delete: Blocked if consumers are still assigned: 'Cannot delete a district with existing consumers.' Must reassign consumers first.

4.6 Staff Management

Maintains personnel records for all barangay water organization staff.

Create: Requires staff ID and name. Role defaults to Meter Reader; status defaults to Active. District and contact are optional.

Read: Lists all staff. Filterable by name, role, or district.

Update: All fields editable including role, district assignment, and status.

Delete: Confirmation: 'Remove this staff member?' Does not affect consumer or billing records.

4.7 Service Requests

Tracks consumer-filed requests (leaks, meter issues, new connections).

Create: Requires consumer. Type defaults to Other; priority defaults to Low. Status is always set to Open on creation.

Read: Lists all requests. Filterable by status (Open / In Progress / Resolved).

Update: Type, priority, and assigned staff are editable. Status can be set to In Progress via edit; use the Resolve button to mark as Resolved.

Delete: Confirmation: 'Delete this service request?' Permanently removes the entry.

5. SETUP & DEPLOYMENT GUIDE

5.1 Prerequisites

- Node.js v18 or higher
- XAMPP (or any MySQL 8.0 server)
- A modern browser (Chrome, Edge, Firefox)
- VS Code with Live Server extension (optional but recommended)

5.2 Database Setup

Step 1. Start XAMPP and ensure the MySQL service is running.

Step 2. Open phpMyAdmin at http://localhost/phpmyadmin

Step 3. Click Import, select backend/db/tapcontrol.sql, and click Go.

This creates the tapcontrol database with all tables and seed data.

5.3 Backend Setup

```
cd backend
npm install
```

Edit backend/.env if your MySQL password is not blank:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD= (your_password_here)
DB_NAME= (tapcontrol)
PORT=3000
```

Start the server:

```
npm start
```

Expected output: TapControl API → http://localhost:3000

5.4 Frontend Setup

No build step is needed. Open any HTML file directly in the browser, or use VS Code Live Server. The frontend JS files are pre-configured to call http://localhost:3000/api/...

6. INDIVIDUAL REFLECTION

Member 1 – Mark Dave G. Asaytuno

Most Difficult Part	Since we had no original author to reference, the hardest part was designing the logic ourselves from scratch. We had to figure out how all the modules connect, for example, we didn't immediately realize that deleting a single consumer needed to cascade through meter readings, billing records, payments, and service requests. That required us to carefully plan the database relationships before we could write any actual code.
Technical Challenge	My biggest challenge was learning multiple new technologies at the same time. I had no prior experience with Node.js and Express, so understanding how routing and middleware worked took a lot of trial and error. Bootstrap was manageable but customizing it with our own CSS sometimes caused conflicts. The hardest part was JavaScript, specifically using the Fetch API to dynamically load and filter table data without page reloads. It was my first time doing that kind of frontend-backend interaction and I spent a lot of time debugging it before getting it to work.
Future Improvements	If I were to improve this system further, I would make the login system more secure since it handles personal data of consumers. I would also add an automated overdue detection so that unpaid bills past their due date are flagged automatically instead of being updated manually. Lastly, a PDF receipt generator would be very useful so staff can print or send payment confirmations directly from the system, which is something the barangay would use on a daily basis.

Member 2 – Joshua Lopera

Most Difficult Part	One of the most difficult parts for me would be exploring new technologies which I have no prior knowledge with. It took some time to connect one thing to another but we manage to hold things together.
Technical Challenge	The technical challenge I encountered during the project would be as I mentioned earlier would be the new technologies that will be applied during the work. Also, when it comes to adding modified CSS for front-end is also challenging when applying.

Future Improvements	For future improvements, it was mentioned also about the login, to be more secure and safer. Larger scale data, notifications whenever you need to pay the bills or bills unpaid.

Member 3 – Aldwin Jude Lunas

Most Difficult Part	One of the most difficult parts for me would be the time management and coordination within the group. There were moments where tasks overlapped and it was hard to keep track of who was doing what, but we managed to communicate and sort things out as a team.
Technical Challenge	The technical challenge I encountered during the project would be setting up the database and making sure all the tables were properly connected to each other. Dealing with relationships between tables and making sure the queries were returning the correct data was something that took a lot of trial and error before getting it right.
Future Improvements	For future improvements, it would be beneficial to add a report generation feature so that administrators can easily view summaries and analytics of the data. Improving the overall user interface to make it more user friendly and visually appealing would also be a great addition, along with adding user roles and permissions to better control access within the system.

Member 4 – Jayson Victor Sapin

Most Difficult Part	The most difficult part for me was understanding how the whole system works from both user and database perspectives. Since the project was built from scratch, we had to build everything by ourselves. It was challenging to ensure every module interacted correctly while still keeping the system simple and easy for barangay staff to use.
Technical Challenge	One of the technical challenges I have experienced was when I was in a push operation; I encountered a Git Bash terminal malfunction. I've somehow fixed it by directly committing the file through the GitHub web interface. A secondary challenge was managing hardware limitations; my laptop's thermal issue caused it to be a bit more laggy, which is frustrating in critical and time-sensitive tasks.
Future Improvements	To improve the system in the future, we should focus on three key updates. First, we need to add a reporting tool so administrators can easily view data summaries and analytics. Second, we should upgrade the user interface to make it more visually appealing and easier to navigate. Finally, introducing user roles and permissions will allow us to better control and secure access within the system.